# The String Conundrum: Understanding and Addressing Novice Programmer Difficulties with String Concepts

Luvuyo Matomela
Department of Computer Science
University of Cape Town
Private Bag X3, Rondebosch, 7701, South Africa
MTMLUV001

**ABSTRACT**

Strings are among the most widely used data types in introductory programming, yet local observation at the University of Cape Town (UCT) suggests that students find string-related tasks disproportionately difficult. This review examines two questions: what does the research literature reveal about the nature of novice programmer difficulties with strings, and what teaching tools have been developed to address those difficulties? The review draws on literature across four areas: the properties of strings that cause confusion, theoretical frameworks for explaining misconceptions (notional machines and threshold concepts), empirical studies of novice programming errors, and educational tools including program visualisers, intelligent tutoring systems, and game-based approaches. The review finds that general programming misconceptions are well documented, but string-specific difficulties remain largely unstudied. Existing program visualisation tools make string behaviour visible but do not teach it. These twin gaps in research and in tooling motivate the development of a dedicated string tutoring tool grounded in the theoretical and empirical foundations identified here.

## 1 INTRODUCTION

Strings are among the most fundamental data types in programming, present in virtually every introductory course as the primary means through which programs interact with text. Despite this ubiquity, strings carry properties that are genuinely non-intuitive for beginners: they are immutable, use zero-based indexing with exclusive end bounds, and behave differently across languages in ways that can produce silent logical errors [15]. Understanding why these properties cause difficulty, and how to address that difficulty through instruction and tooling, is the central concern of this review.

The motivation for this review is practical. Observation at UCT suggests that students find string-related tasks disproportionately difficult compared to other introductory topics. Yet the existing literature does not reflect this: studies of novice programming errors focus predominantly on variables, loops, conditionals, and scope, with strings appearing only in passing [15], [16]. This tension between a locally observed difficulty and an absence of dedicated research makes the question of whether strings constitute a distinct and significant problem an open one, and makes this research necessarily exploratory.

Addressing this gap requires two complementary lines of inquiry. The first concerns the research base: what is known about how and why novice programmers struggle with programming concepts in general, and with strings in particular? The second concerns tooling: what educational tools have been developed to support novice programmers, what evidence exists for their effectiveness, and why does none of them specifically target string concepts? These two strands are not independent the theoretical frameworks that explain misconceptions directly inform what a string-focused teaching tool should do, and the methodological lessons from existing tool evaluations shape how such a tool should be assessed.

The review is scoped to Python and Java, the two languages most commonly used in first-year university programming courses. Section 2 provides background on the string properties most likely to cause difficulties. Section 3 introduces the theoretical frameworks used in computing education research to explain why misconceptions form and persist. Section 4 surveys the empirical evidence on novice programmer errors, with particular attention to what is and is not known about strings. Section 5 reviews the three main categories of educational tool for novice programmers. Section 6 synthesises the prior work, compares the approaches, and identifies the gaps this research addresses. Section 7 concludes.

## 2 BACKGROUND: STRINGS IN INTRODUCTORY PROGRAMMING

### *2.1 String Properties as Sources of Difficulty*

Three string properties come up consistently as sources of difficulty for beginner programmers: immutability, zero-based indexing, and type-related differences between languages [15].

In Python and Java, strings cannot be changed once they are created. If you do something like add text to a string, the program creates a brand new string rather than changing the old one. Robins et al. [16] document a widespread novice tendency to treat variables as containers whose contents change in place, meaning students expect that performing an operation will modify the existing value directly. Applied to strings, this leads to a common bug where a student does a string operation but forgets to save the result, not realising that the original variable did not change.

Zero-based indexing is another source of confusion. Most people count starting from one, but programming starts from zero. Qian and Lehman [15] identify off-by-one errors as a recurring difficulty in the category of conceptual knowledge mistakes, where students' prior intuitions about counting conflict with the conventions of the language. Python also has slice notation where the end index is not included, which adds another layer of potential

confusion if the convention is not made explicit during teaching [15].

In Java, using == to compare two strings checks whether they are the same object in memory, not whether they have the same text. Students used to comparing numbers with == apply the same logic to strings and end up with bugs that are hard to spot [12]. Python does not have this exact problem but has its own quirks around how strings are stored and encoded [15].

### *2.2 How Strings Are Taught*

Guzdial [7] notes that strings are often introduced early in courses just to get students printing and reading text, then largely forgotten as the course moves on to loops, conditions, and functions. This means the trickier parts of strings, like immutability and indexing, get covered before students know enough to really understand them. Robins et al. [16] highlight that novice knowledge is fragile: students develop partial, context-dependent understandings of programming concepts that can break down when a concept is encountered in a new context. Clancy and Linn [4] argue that effective programming instruction requires deliberate integration of related concepts rather than treating them as isolated topics students need to encounter the same underlying idea in multiple contexts before it becomes robust. Introducing strings early without returning to their deeper properties later means that the kind of integration Clancy and Linn identify as necessary for durable understanding is unlikely to occur [4], [16].

## 3 THEORETICAL FRAMEWORKS

### *3.1 Notional Machines*

One of the main ideas used in computing education research is the notional machine, a concept introduced by du Boulay [1] to describe the simplified, abstract model of a computer that a student constructs in order to reason about program execution. It is not intended to be technically complete, only useful enough to support correct reasoning about what a given program will do [1], [17]. Problems arise when a student's notional machine does not accurately reflect the language's actual behaviour when their internal model predicts one outcome and the program produces another [17].

Sorva [17] argues that when students get things wrong, it is not always because they do not know enough. Often they have built up a mental model that is wrong but feels right to them. For example, a student who thinks strings can be changed in place will keep making the same mistake even after being told the correct answer. Fixing this takes more than just explaining it again. The student needs to see something that shows them clearly that their model is wrong [17].

### *3.2 Threshold Concepts*

Another useful idea is the threshold concept, introduced by Meyer and Land [11] and applied to computing education by Boustedt et al. [3]. A threshold concept is an idea that, once you really understand it, transforms the way a student thinks about the discipline. Boustedt et al. found empirical evidence that object-oriented programming and pointers function as threshold concepts in computer science: they are transformative, integrative, and difficult to unlearn once understood [3]. The same criteria plausibly apply to string immutability. Before a student truly grasps that string operations produce new objects rather than modifying the original, their understanding of how assignment and memory work in Python and Java remains incomplete. Grasping immutability shifts that understanding in a way that matches the transformative and integrative qualities that define a threshold concept [3], [11].

This matters for tool design. If immutability is the kind of idea that is hard to grasp just by reading about it, then a tool that makes the concept visible, by actually showing the student that a new string has been created, is likely to be more effective than a written explanation [8].

### *3.3 Why Misconceptions Are Hard to Fix*

Pea [14] described a pattern he called superbug errors, where beginner programmers act as if the computer can read their mind. They expect the program to do what they meant, not exactly what they wrote. With strings, this shows up as students expecting a string to update itself when they operate on it. Qian and Lehman [15] confirm that inaccurate mental models remain one of the most persistent sources of difficulty in introductory programming, suggesting that the underlying dynamic Pea identified has continued relevance for understanding why novices struggle.

Murphy et al. [6] found in a study across multiple universities that many novice programmers applied debugging strategies ineffectively or engaged in unproductive behaviours even after encountering errors. Some students used few strategies and simply repeated the same approach, leading to frustration and, in some cases, the introduction of new bugs. This behavioural pattern is consistent with what the notional machine framework predicts: a student whose internal model of program execution is wrong will not be able to reason correctly about what is failing, and correcting the surface behaviour without correcting the model leaves the underlying problem intact. The implication is that effective instruction must make the model not just the error visible to the student [6], [17].

## 4 NOVICE PROGRAMMER MISCONCEPTIONS

### *4.1 What the General Research Shows*

The literature on novice programming errors is extensive. Qian and Lehman [15] synthesised decades of research and identified the most prevalent difficulty areas as variables and assignment, loops and conditionals, scope, and data types. Robins et al. [16] offer a complementary taxonomy, distinguishing syntax errors which the compiler surfaces immediately from semantic errors, which produce incorrect behaviour without any error message. Taken together, these reviews suggest that the errors most likely to persist and most difficult to remediate are those in the semantic category, where the student's mental model of what the program is doing diverges from its actual execution without any automatic signal that something is wrong. Strings fall squarely into this category: string operations rarely cause syntax errors, but the absence of a visible error makes the underlying conceptual problem harder to detect and correct [15], [16].

What the general literature does not do is examine string-related errors in isolation. The following section considers what limited string-specific evidence does exist, and what can be inferred from the broader research.

### *4.2 What the Research Says About Strings Specifically*

There is not much research that looks at string mistakes directly. Unlike topics like variables or recursion, which have been studied a lot, strings usually only show up as a footnote in studies focused on something else [15], [16]. The lack of dedicated research on strings is itself an important finding for this review.

Murphy et al. [6] studied novice Java programmers across multiple universities and found that many struggled to debug programs effectively, often applying strategies ineffectively or abandoning them too quickly. A recurring difficulty was students' inability to reason about what was happening in memory at each step of execution. Milne and Rowe [12] found that the programming topics students and tutors rated as hardest to learn were precisely those that depend on a clear mental model of memory: pointers, dynamic allocation, and related concepts scored highest on their difficulty scale. This difficulty with memory visibility is directly relevant to strings, since understanding why a string operation does not change the original variable requires students to reason about how objects are stored and referenced in memory [12].

Qian and Lehman [15] identify inaccurate mental models as one of the central factors contributing to persistent difficulties in introductory programming. They note that these conceptual difficulties are harder to address than surface-level syntax errors because they do not produce immediate, visible feedback. This is exactly the situation with string operations: the program runs and produces output, but the output may be wrong in ways that only become visible if the student can reason carefully about how values are stored and updated [15].

The limited research on strings lines up with what the research supervisor noted: strings are not a well-known problem in the literature. The argument for strings being a real issue has to come from the theoretical analysis in Section 2, the indirect evidence in the studies above, and the local observation at UCT.

## 5 EDUCATIONAL TOOLS FOR NOVICE PROGRAMMERS

### *5.1 Program Visualisation*

The most well-researched type of teaching tool in computing education is program visualisation. Guo [8] built Online Python Tutor, a website where students can step through a Python or Java program one line at a time and see what every variable and object looks like at each step. It has been used in courses around the world and studied in a number of evaluations [8].

Sorva et al. [18] reviewed more than thirty of these kinds of tools built between 1975 and 2013. They found that students generally like them and rate them as useful. When students use them as part of a structured activity with specific tasks, they tend to learn more. But when students just use them to explore on their own, the results are less consistent. The main takeaway is that the tool alone is not enough; how it is used matters a lot [18].

Importantly, none of the tools in Sorva et al.'s review [18] was built with the goal of teaching strings specifically. Python Tutor does show string objects, and a student can see that a new string is created during concatenation. But there is nothing in the tool that points this out, explains why it happens, or gives the student a task built around it. The string behaviour is visible but not taught [8], [18].

### *5.2 Intelligent Tutoring Systems*

Intelligent tutoring systems (ITS) are tools that adjust what they teach based on how well a student is doing. VanLehn [19] conducted a meta-analytic review comparing ITS, human one-on-one tutoring, and no tutoring. Contrary to widely-held beliefs, he found that human tutoring produced an effect size of $d = 0.79$ rather than the often-cited $d = 2.0$, and that ITS produced an effect size of $d = 0.76$ nearly equivalent to human tutoring when both are compared to instruction without tutoring. The central advantage of both forms of tutoring is that they give students timely, targeted feedback that instruction alone does not [19].

Corbett and Anderson [5] developed a method called knowledge tracing, which is widely used in ITS design. It tracks how well a student knows each specific concept and uses that to decide what to show them next. For a string tutoring tool, this could mean tracking whether a student understands indexing, immutability, and type comparison separately, and adjusting the difficulty of tasks based on that. No existing ITS has been found that does this for string concepts [5], [19].

### *5.3 Game-Based and Interactive Tools*

Leutenegger and Edgington [10] redesigned an introductory programming sequence using 2D game development as a unifying theme. Their rationale was that compelling, visual assignments increase student motivation and allow students to see the effects of errors directly in the resulting animation. Their surveys showed improvements in student understanding across all seven core programming topics they examined. The key insight is that immediate visual feedback seeing a game behave incorrectly when code contains a bug makes the connection between a mistake and its consequence concrete in a way that abstract exercises do not [10].

This idea applies directly to string teaching. If a tool shows a student what happens to a string after each operation, including creating a new object when the student expected the old one to change, the student gets instant feedback that something is not working the way they thought. This kind of feedback is exactly what research says helps students fix wrong mental models [6], [17].

### *5.4 User-Centred Design and Evaluation*

Preece, Rogers, and Sharp [13] describe a design approach where the people who will use the tool are involved throughout the whole development process, from the start right through to testing finished versions. For a student-facing tool, this means testing with actual beginner programmers at every stage, not just at the end. They separate two goals: making the tool easy to use, and making sure students actually learn from it.

Ko, Latoza, and Burnett [9] give practical advice on how to run proper evaluations of software tools with real users. They highlight common mistakes in tool research, like not checking how much students already knew before the study, or measuring how easy the tool was to use instead of how much students learned. Their advice on measuring both process (e.g. error rate while using the tool) and outcome (e.g. test scores before and after) will shape how the proposed system gets evaluated.

## 6 CRITICAL COMPARISON AND SYNTHESIS

### *6.1 What the Research Agrees On*

A few clear patterns come out across all the research reviewed. First, beginner programming mistakes tend to stick around. Telling students what they got wrong is often not enough to fix it; they need an experience that shows them the gap between what they expected and what actually happened [6], [15]. Second, the best tool for creating that experience is visualisation: showing students the inside of a program as it runs gives them something concrete to compare against their expectations [17], [18]. Third, visualisation

works better when there is structure around it. Open-ended exploration helps less than specific guided tasks [18].

### *6.2 Where the Research Falls Short*

The biggest gap is that string mistakes have barely been studied on their own. Topics like conceptual bugs in general [14], recursion [2], and object-oriented programming concepts such as pointers and memory management [12] each have dedicated research examining the specific nature of student difficulties. String handling, by contrast, does not. Milne and Rowe [12] included string handling as one of twenty-eight topics in their difficulty survey, where it received a mid-range difficulty score present, but not flagged as a primary concern. No study has examined the full range of string-specific mistakes in a careful, language-focused way [15], [16].

There is also no tool built specifically to teach strings. Visualisers like Python Tutor [8] show string behaviour as part of a general view of program execution, but there is no guidance built in to help students notice and understand the string-specific things that happen [18]. ITS approaches [5], [19] could in theory be applied to strings, but no one has done it.

A third problem is with how tools have been evaluated. Ko et al. [9] point out that many studies mix up ease of use with learning, and do not control for how much students already knew going in. This means claims about how well tools like Python Tutor [8] improve learning should be taken with some caution.

### *6.3 What This Research Does About It*

This research addresses two of those three gaps directly. For the empirical gap, it will gather evidence at UCT about whether strings genuinely constitute a more significant source of difficulty than other topics, drawing on student assessment data and, if ethical clearance permits, anonymised performance records. This local evidence is necessary because the absence of string-specific findings in the broader literature cannot be taken as evidence of absence of difficulty it may simply reflect that no one has looked carefully yet. For the tool gap, the review provides a principled foundation for design. The notional machine framework [17] establishes that string operations must be made visually explicit: students need to see that immutability produces a new object, not because they are told so, but because they can observe it directly. The threshold concept framework [3] establishes that immutability should be treated as a gateway idea and sequenced accordingly not introduced incidentally but foregrounded and returned to. Knowledge tracing [5] provides the mechanism for personalisation, allowing the tool to track whether each student has consolidated understanding of indexing, immutability, and type comparison as distinct concepts and to adapt task difficulty accordingly.

The methodological gap identified by Ko et al. [9] will be addressed by designing the evaluation to avoid the pitfalls they document: pre- and post-testing will establish a baseline and measure learning gain separately from usability; time on task will be controlled; and the study will distinguish between students finding the tool easy to use and students actually learning from it. User-centred design principles [13] will be applied throughout the development process, with beginner programmers involved at each stage rather than only at the end. This approach means the tool is shaped by real student behaviour, not assumptions about it.

### *6.4 Comparing the Theoretical Frameworks*

The notional machine [17] and threshold concept [3], [11] frameworks presented in Section 3 are complementary rather than competing. The notional machine is primarily descriptive: it explains why students develop incorrect mental models and why those models resist correction. Threshold concepts are prescriptive: they identify which ideas deserve the most deliberate instructional attention because misunderstanding them has the widest knock-on effects [3]. For string teaching, this means that immutability should not be treated as just another fact to memorise. It is a gateway idea, and the research reviewed here suggests that students need to encounter it in a way that makes the gap between their expectation and reality concrete and impossible to ignore [17].

Pea's superbug framework [14] and the findings of Murphy et al. [6] connect in a similarly productive way. Pea identified the diagnostic pattern: students expect programs to do what they intended rather than what they wrote [14]. Murphy et al. documented the behavioural consequence: novice debuggers often applied strategies ineffectively and repeated unproductive behaviours even when their attempts were not working [6]. Together these two bodies of work point to a clear implication: passive correction does not work, and something more active is required. The visualisation research in Section 5 provides exactly that mechanism. By making program execution visible one step at a time, tools like Python Tutor [8] create the kind of confrontational feedback that Sorva [17] argues is necessary to dislodge a wrong model. The limitation is that none of this has been targeted at strings specifically [8], [18].

A productive tension emerges between the theoretical frameworks reviewed in Section 3 and the tool literature reviewed in Section 5. The notional machine and threshold concept frameworks are grounded in the observation that certain misconceptions are near-universal: most students form the same wrong model of string immutability regardless of background, language experience, or prior instruction [3], [17]. This suggests that a string tutoring tool can be designed around a shared conceptual core a fixed set of target ideas that nearly all students need to confront. The ITS literature, however, starts from the opposite premise: that learners differ significantly in what they already know and at what rate they progress, and that adaptive personalisation through knowledge tracing is what makes the approach effective [5], [19]. These positions are not in conflict, but they operate at different levels of grain. The universal-misconception view shapes what the tool teaches; the adaptive-personalisation view shapes how and when it teaches it. A well-designed string tutoring tool must be grounded in both: built around the predictable conceptual obstacles that theory and evidence identify [14], while remaining responsive to each student's actual trajectory rather than assuming a uniform path through the material [5].

### *6.5 Comparing the Tool Approaches*

The three categories of teaching tool reviewed in Section 5 each have distinct strengths and limitations. Program visualisers like Python Tutor [8] have the broadest evidence base and the widest real-world adoption. Their strength is generality: any program can be stepped through and any variable inspected [8]. Their weakness, for the purposes of this research, is precisely that same generality. A student using Python Tutor to work through a string operation still has to notice on their own that a new object was created, reason about why it matters, and connect it back to whatever mental model they were working from. The tool makes behaviour visible but does not teach it [8], [18]. This distinction between making something

visible and making it understood is central to what this research is trying to improve on.

Intelligent tutoring systems [5], [19] address the guidance problem but at a different cost. ITS platforms are highly effective because they close the feedback loop: they track which concepts a student has and has not mastered, and adjust what comes next accordingly [5]. VanLehn [19] found that ITS produces learning gains nearly equivalent to one-on-one human tutoring, with both achieving substantially better outcomes than instruction without any tutoring component. The problem is that building an ITS requires a well-defined knowledge model for the target domain [5]. For string concepts, no such model has been published, meaning any string-focused ITS would need to construct that model from scratch. The ITS literature therefore provides valuable design principles rather than a ready-made foundation.

Game-based and interactive tools [10] sit between these two approaches. They are more engaging than static step-through visualisers and cheaper to build than full ITS platforms [10]. The tradeoff is that engagement does not automatically translate into learning. Sorva et al. [18] found that open-ended exploration with visualisation tools produced less consistent learning gains than structured activities with guided questions. This finding matters for design: it is not enough to build a tool that shows string operations in an interesting way. The tool must also prompt students to predict what will happen, check whether their prediction was right, and reflect on the gap when it was not [17], [18]. These active learning features are what move a tool from being a visual aid to being a genuine tutoring instrument.

One further point cuts across all three tool categories. Ko et al. [9] raise serious methodological concerns about how most of these tools have been assessed: studies often conflate ease of use with learning gain, fail to measure prior knowledge, or rely on self-reported satisfaction as a proxy for educational effectiveness [9]. This means the evidence base for all three tool types is weaker than it appears. A well-designed evaluation that measures pre- and post-test knowledge separately, controls for time on task, and distinguishes usability from learning, would be a meaningful contribution in itself [9], [13]. The evaluation design for this research will be shaped directly by these concerns.

## 7 CONCLUSION

This review addressed two questions: whether the research literature identifies strings as a significant source of difficulty for novice programmers, and whether any dedicated tools exist to address that difficulty. The answer to both is negative, but in a way that is informative rather than merely limiting.

String-specific difficulties are not well represented in the computing education literature, and no tool designed specifically to teach string concepts was identified. These absences confirm that this research occupies genuinely unexplored ground. They also mean that the case for string difficulty cannot rest on direct evidence alone: it must be constructed from theoretical prediction, indirect empirical support, and local observation which is precisely the structure this review has provided.

The review has also identified a substantial foundation to build on. The notional machine and threshold concept frameworks provide a principled account of why string misconceptions form, why they persist, and which concepts should receive the most deliberate instructional attention. The literature on program visualisation, intelligent tutoring systems, and game-based approaches provides concrete design principles and methodological lessons. User-centred design and evaluation methodology provides the means to build and assess the tool rigorously.

The research is exploratory in the sense that its first task is to establish whether strings constitute a distinct and significant difficulty at UCT. If they do, the theoretical and empirical groundwork assembled here provides a clear basis for the design of a tutoring tool that addresses those difficulties in a principled way. If the evidence is less clear-cut, this review still contributes by establishing the conceptual framework within which any future string-focused intervention should be understood.

## References


[1] du Boulay, B. 1986. Some difficulties of learning to program. Journal of Educational Computing Research, 2, 1, 57-73. https://doi.org/10.2190/3LFX-9RRF-67T8-UVK9

[2] Ginat, D. and Shifroni, E. 1999. Teaching recursion in a procedural environment how much should we emphasize the computing model? In Proceedings of the 30th SIGCSE Technical Symposium on Computer Science Education. ACM, 127–131. https://doi.org/10.1145/299649.299718

[3] Boustedt, J., Eckerdal, A., McCartney, R., Mostrom, J.E., Ratcliffe, M., Sanders, K., and Zander, C. 2007. Threshold concepts in computer science: do they exist and are they useful? In Proceedings of the 38th SIGCSE Technical Symposium on Computer Science Education. ACM, 504-508. https://doi.org/10.1145/1227310.1227482

[4] Clancy, M. and Linn, M. 1999. Patterns and pedagogy. In Proceedings of the 30th SIGCSE Technical Symposium on Computer Science Education. ACM, 37-42. https://doi.org/10.1145/299649.299673

[5] Corbett, A.T. and Anderson, J.R. 1994. Knowledge tracing: Modelling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4, 4, 253-278. https://doi.org/10.1007/BF01099821

[6] Murphy, L., Lewandowski, G., McCauley, R.., Simon, B., Thomas, L., and Zander, C. 2008. Debugging: the good, the bad, and the quirky -- a qualitative analysis of novices' strategies. Computer Science Education, 18, 2, 93-116. https://dl.acm.org/doi/10.1145/1352135.1352191

[7] Guzdial, M. 2015. Learner-Centered Design of Computing Education: Research on Computing for Everyone. Morgan & Claypool, San Rafael, CA.

[8] Guo, P.J. 2013. Online Python Tutor: embeddable web-based program visualization for CS education. In Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13). ACM, 579-584. https://doi.org/10.1145/2445196.2445368

[9] Ko, A.J., Latoza, T.D., and Burnett, M.M. 2015. A practical guide to controlled experiments of software engineering tools with human participants. Empirical Software Engineering, 20, 1, 110-141. https://doi.org/10.1007/s10664-013-9279-3

[10] Leutenegger, S. and Edgington, J. 2007. A games first approach to teaching introductory programming. In Proceedings of the 38th SIGCSE Technical Symposium on Computer Science Education. ACM, 115-118. https://doi.org/10.1145/1227310.1227352

[11] Meyer, J.H.F. and Land, R. 2003. Threshold concepts and troublesome knowledge: linkages to ways of thinking and practising within the disciplines. In C. Rust (Ed.), Improving Student Learning: Theory and Practice Ten Years On. Oxford Centre for Staff and Learning Development, Oxford, 412-424.

[12] Milne, I. and Rowe, G. 2002. Difficulties in learning and teaching programming: views of students and tutors. Education and Information Technologies, 7, 1, 55-66. https://doi.org/10.1023/A:1015362608943

[13] Preece, J., Rogers, Y., and Sharp, H. 2015. Interaction Design: Beyond Human-Computer Interaction. 4th ed. Wiley, Chichester, UK.

[14] Pea, R.D. 1986. Language-independent conceptual bugs in novice programming. Journal of Educational Computing Research, 2, 1, 25-36. https://doi.org/10.2190/689T-1R2A-X4W4-29TM

[15] Qian, Y. and Lehman, J. 2017. Students' misconceptions and errors in programming: a review of the literature ACM Transactions on Computing Education, 18, 1, Article 1. https://doi.org/10.1145/3077618

[16] Robins, A., Rountree, J., and Rountree, N. 2003. Learning and teaching programming: a review and discussion. Computer Science Education, 13, 2, 137-172. https://doi.org/10.1076/csed.13.2.137.14200

[17] Sorva, J. 2012. Visual program simulation in introductory programming education. Doctoral dissertation, Aalto University, Finland.

[18] Sorva, J., Karavirta, V., and Malmi, L. 2013. A review of generic program visualization systems for introductory programming education. ACM Transactions on Computing Education, 13, 4, 1-64. https://doi.org/10.1145/2490822

[19] VanLehn, K. 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist, 46, 3, 197-221. https://doi.org/10.1080/00461520.2011.611369